

Creating a Problem Statement

1. Understand the dataset:

1.1 Import the dataset

```
In [67]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
sr=pd.read_csv("311_Service_Requests_from_2010_to_Present.csv")
```

C:\Users\ganesh\AppData\Local\Temp\ipykernel_17692\3130146605.py:6: DtypeWarning: Columns (0: Ferry Direction, 1: Ferry Terminal Name) have mixed types. Specify dtype option on import or set low_memory=False.

```
sr=pd.read_csv("311_Service_Requests_from_2010_to_Present.csv")
```

1.2 Visualize the dataset

```
In [2]: sr
```

Out[2]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Description
0	32310363	12/31/2015 11:59:45 PM	01/01/2016 12:55:15 AM	NYPD	New York City Police Department	Noise - Street/Sidewalk	Loud Music/Party
1	32309934	12/31/2015 11:59:44 PM	01/01/2016 01:26:57 AM	NYPD	New York City Police Department	Blocked Driveway	No Access
2	32309159	12/31/2015 11:59:29 PM	01/01/2016 04:51:03 AM	NYPD	New York City Police Department	Blocked Driveway	No Access
3	32305098	12/31/2015 11:57:46 PM	01/01/2016 07:43:13 AM	NYPD	New York City Police Department	Illegal Parking	Commercial Overnight Parking
4	32306529	12/31/2015 11:56:58 PM	01/01/2016 03:24:42 AM	NYPD	New York City Police Department	Illegal Parking	Blocked Sidewalk
...	...	...	...	...	...	...	...
364553	29609918	01/01/2015 12:04:44 AM	01/01/2015 10:22:31 AM	NYPD	New York City Police Department	Illegal Parking	Blocked Hydrant
364554	29608392	01/01/2015 12:04:28 AM	01/01/2015 02:25:02 AM	NYPD	New York City Police Department	Noise - Vehicle	Car/Truck Horn
364555	29607589	01/01/2015 12:01:30 AM	01/01/2015 12:20:33 AM	NYPD	New York City Police Department	Noise - Street/Sidewalk	Loud Music/Party
364556	29610889	01/01/2015 12:01:29 AM	01/01/2015 02:42:22 AM	NYPD	New York City Police Department	Blocked Driveway	No Access
364557	29611816	01/01/2015 12:00:50 AM	01/01/2015 02:47:50 AM	NYPD	New York City Police Department	Blocked Driveway	No Access

364558 rows × 53 columns



In [3]: `sr.tail()`

Out[3]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Description
364553	29609918	01/01/2015 12:04:44 AM	01/01/2015 10:22:31 AM	NYPD	New York City Police Department	Illegal Parking	Blocked Hydrant
364554	29608392	01/01/2015 12:04:28 AM	01/01/2015 02:25:02 AM	NYPD	New York City Police Department	Noise - Vehicle	Car/Truck Horn
364555	29607589	01/01/2015 12:01:30 AM	01/01/2015 12:20:33 AM	NYPD	New York City Police Department	Noise - Street/Sidewalk	Loud Music/Party
364556	29610889	01/01/2015 12:01:29 AM	01/01/2015 02:42:22 AM	NYPD	New York City Police Department	Blocked Driveway	No Access
364557	29611816	01/01/2015 12:00:50 AM	01/01/2015 02:47:50 AM	NYPD	New York City Police Department	Blocked Driveway	No Access

5 rows x 53 columns



1.3 Print the columns of the DataFrame

In [4]: `sr.columns`

```
Out[4]: Index(['Unique Key', 'Created Date', 'Closed Date', 'Agency', 'Agency Name',  
              'Complaint Type', 'Descriptor', 'Location Type', 'Incident Zip',  
              'Incident Address', 'Street Name', 'Cross Street 1', 'Cross Street 2',  
              'Intersection Street 1', 'Intersection Street 2', 'Address Type',  
              'City', 'Landmark', 'Facility Type', 'Status', 'Due Date',  
              'Resolution Description', 'Resolution Action Updated Date',  
              'Community Board', 'Borough', 'X Coordinate (State Plane)',  
              'Y Coordinate (State Plane)', 'Park Facility Name', 'Park Borough',  
              'School Name', 'School Number', 'School Region', 'School Code',  
              'School Phone Number', 'School Address', 'School City', 'School State',  
              'School Zip', 'School Not Found', 'School or Citywide Complaint',  
              'Vehicle Type', 'Taxi Company Borough', 'Taxi Pick Up Location',  
              'Bridge Highway Name', 'Bridge Highway Direction', 'Road Ramp',  
              'Bridge Highway Segment', 'Garage Lot Name', 'Ferry Direction',  
              'Ferry Terminal Name', 'Latitude', 'Longitude', 'Location'],  
              dtype='str')
```

1.4 Identify the shape of the dataset

In [5]: `sr.shape`

Out[5]: (364558, 53)

1.5 Identify the variables with null values

In [6]: `sr.info()`

<class 'pandas.DataFrame'>

RangeIndex: 364558 entries, 0 to 364557

Data columns (total 53 columns):

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	Unique Key	364558 non-null	int64
1	Created Date	364558 non-null	str
2	Closed Date	362177 non-null	str
3	Agency	364558 non-null	str
4	Agency Name	364558 non-null	str
5	Complaint Type	364558 non-null	str
6	Descriptor	358057 non-null	str
7	Location Type	364425 non-null	str
8	Incident Zip	361560 non-null	float64
9	Incident Address	312859 non-null	str
10	Street Name	312859 non-null	str
11	Cross Street 1	307370 non-null	str
12	Cross Street 2	306753 non-null	str
13	Intersection Street 1	51120 non-null	str
14	Intersection Street 2	50512 non-null	str
15	Address Type	361306 non-null	str
16	City	361561 non-null	str
17	Landmark	375 non-null	str
18	Facility Type	362169 non-null	str
19	Status	364558 non-null	str
20	Due Date	364555 non-null	str
21	Resolution Description	364558 non-null	str
22	Resolution Action Updated Date	362156 non-null	str
23	Community Board	364558 non-null	str
24	Borough	364558 non-null	str
25	X Coordinate (State Plane)	360528 non-null	float64
26	Y Coordinate (State Plane)	360528 non-null	float64
27	Park Facility Name	364558 non-null	str
28	Park Borough	364558 non-null	str
29	School Name	364558 non-null	str
30	School Number	364558 non-null	str
31	School Region	364557 non-null	str
32	School Code	364557 non-null	str
33	School Phone Number	364558 non-null	str
34	School Address	364558 non-null	str
35	School City	364558 non-null	str
36	School State	364558 non-null	str
37	School Zip	364557 non-null	str
38	School Not Found	364558 non-null	str
39	School or Citywide Complaint	0 non-null	float64
40	Vehicle Type	0 non-null	float64
41	Taxi Company Borough	0 non-null	float64
42	Taxi Pick Up Location	0 non-null	float64
43	Bridge Highway Name	297 non-null	str
44	Bridge Highway Direction	297 non-null	str
45	Road Ramp	262 non-null	str
46	Bridge Highway Segment	262 non-null	str
47	Garage Lot Name	0 non-null	float64
48	Ferry Direction	1 non-null	str
49	Ferry Terminal Name	2 non-null	str
50	Latitude	360528 non-null	float64

```

51 Longitude                                360528 non-null float64
52 Location                                360528 non-null str
dtypes: float64(10), int64(1), str(42)
memory usage: 147.4 MB

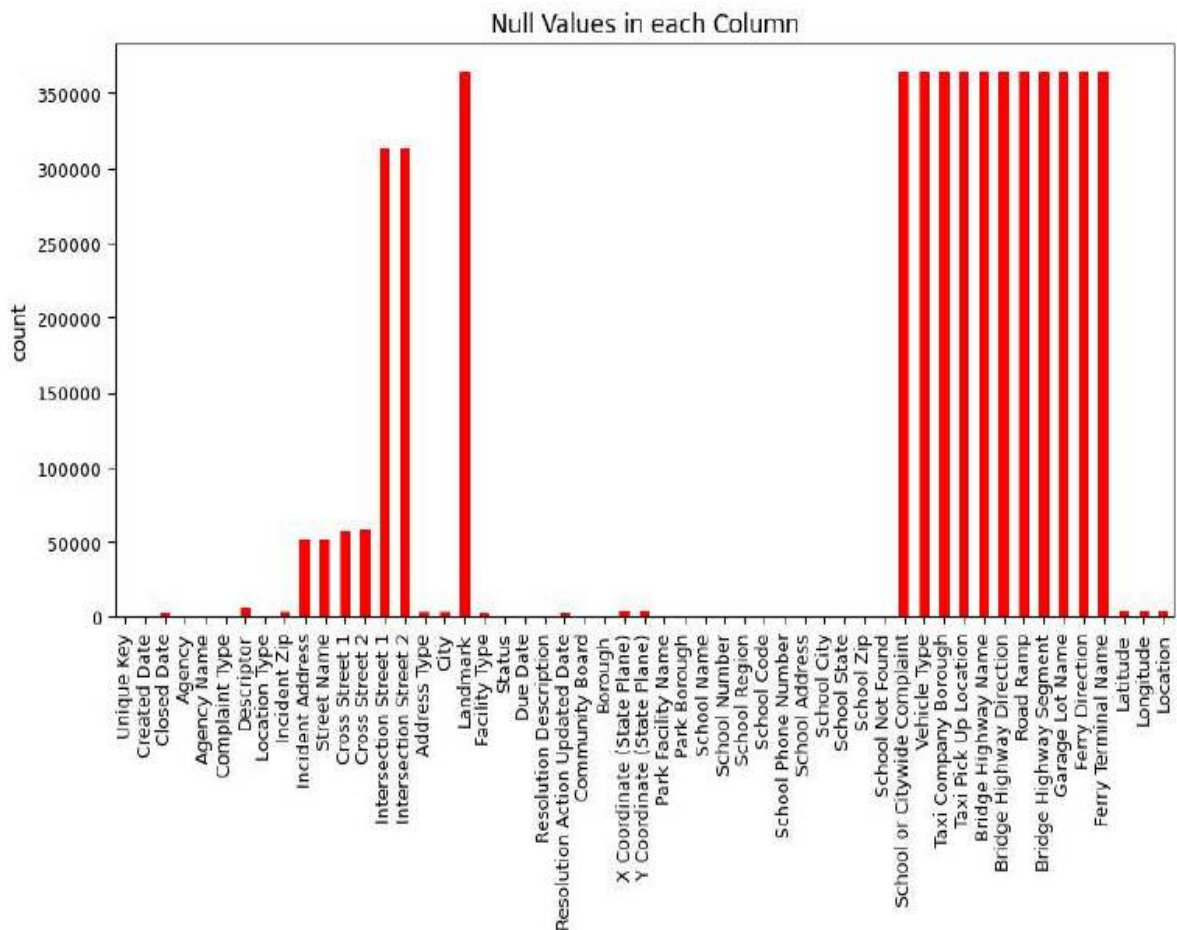
```

2. Perform basic data exploratory analysis:

2.1 Draw a frequency plot to show the number of null values in each column of the DataFrame

```
In [7]: null_count= sr.isnull().sum()
```

```
In [68]: ##plt.figure(figsize=(5,4))
plt.title('Null Values in each Column')
null_count.plot(kind='bar',figsize=(10,5),color="red",fontsize=9)
plt.ylabel("count");
```



2.2 Missing value treatment

2.2.1 Remove the records whose Closed Date values are null

```
In [9]: sr=sr[sr['Closed Date'].notnull()]
sr
```

Out[9]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Description
0	32310363	12/31/2015 11:59:45 PM	01/01/2016 12:55:15 AM	NYPD	New York City Police Department	Noise - Street/Sidewalk	Loud Music/Party
1	32309934	12/31/2015 11:59:44 PM	01/01/2016 01:26:57 AM	NYPD	New York City Police Department	Blocked Driveway	No Access
2	32309159	12/31/2015 11:59:29 PM	01/01/2016 04:51:03 AM	NYPD	New York City Police Department	Blocked Driveway	No Access
3	32305098	12/31/2015 11:57:46 PM	01/01/2016 07:43:13 AM	NYPD	New York City Police Department	Illegal Parking	Commercial Overnight Parking
4	32306529	12/31/2015 11:56:58 PM	01/01/2016 03:24:42 AM	NYPD	New York City Police Department	Illegal Parking	Blocked Sidewalk
...	...	...	...	...	...	...	...
364553	29609918	01/01/2015 12:04:44 AM	01/01/2015 10:22:31 AM	NYPD	New York City Police Department	Illegal Parking	Blocked Hydrant
364554	29608392	01/01/2015 12:04:28 AM	01/01/2015 02:25:02 AM	NYPD	New York City Police Department	Noise - Vehicle	Car/Truck Horn
364555	29607589	01/01/2015 12:01:30 AM	01/01/2015 12:20:33 AM	NYPD	New York City Police Department	Noise - Street/Sidewalk	Loud Music/Party
364556	29610889	01/01/2015 12:01:29 AM	01/01/2015 02:42:22 AM	NYPD	New York City Police Department	Blocked Driveway	No Access
364557	29611816	01/01/2015 12:00:50 AM	01/01/2015 02:47:50 AM	NYPD	New York City Police Department	Blocked Driveway	No Access

362177 rows × 53 columns



2.3 Analyze the date column, and remove entries that have an incorrect timeline

2.3.1 Calculate the time elapsed in closed and creation date

```
In [10]: sr['Created Date']=pd.to_datetime(sr['Created Date'],errors='coerce')
```

```
C:\Users\ganesh\AppData\Local\Temp\ipykernel_17692\3056093407.py:1: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.  
sr['Created Date']=pd.to_datetime(sr['Created Date'],errors='coerce')
```

```
In [11]: sr['Closed Date']=pd.to_datetime(sr['Closed Date'],errors='coerce')
```

```
C:\Users\ganesh\AppData\Local\Temp\ipykernel_17692\1156337200.py:1: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.  
sr['Closed Date']=pd.to_datetime(sr['Closed Date'],errors='coerce')
```

```
In [12]: sr.info()
```

```
<class 'pandas.DataFrame'>
Index: 362177 entries, 0 to 364557
Data columns (total 53 columns):
```

#	Column	Non-Null Count	Dtype
0	Unique Key	362177 non-null	int64
1	Created Date	362177 non-null	datetime64[us]
2	Closed Date	362177 non-null	datetime64[us]
3	Agency	362177 non-null	str
4	Agency Name	362177 non-null	str
5	Complaint Type	362177 non-null	str
6	Descriptor	355681 non-null	str
7	Location Type	362047 non-null	str
8	Incident Zip	361502 non-null	float64
9	Incident Address	310491 non-null	str
10	Street Name	310491 non-null	str
11	Cross Street 1	306846 non-null	str
12	Cross Street 2	306713 non-null	str
13	Intersection Street 1	50628 non-null	str
14	Intersection Street 2	50504 non-null	str
15	Address Type	361248 non-null	str
16	City	361503 non-null	str
17	Landmark	375 non-null	str
18	Facility Type	362159 non-null	str
19	Status	362177 non-null	str
20	Due Date	362176 non-null	str
21	Resolution Description	362177 non-null	str
22	Resolution Action Updated Date	362138 non-null	str
23	Community Board	362177 non-null	str
24	Borough	362177 non-null	str
25	X Coordinate (State Plane)	360470 non-null	float64
26	Y Coordinate (State Plane)	360470 non-null	float64
27	Park Facility Name	362177 non-null	str
28	Park Borough	362177 non-null	str
29	School Name	362177 non-null	str
30	School Number	362177 non-null	str
31	School Region	362176 non-null	str
32	School Code	362176 non-null	str
33	School Phone Number	362177 non-null	str
34	School Address	362177 non-null	str
35	School City	362177 non-null	str
36	School State	362177 non-null	str
37	School Zip	362176 non-null	str
38	School Not Found	362177 non-null	str
39	School or Citywide Complaint	0 non-null	float64
40	Vehicle Type	0 non-null	float64
41	Taxi Company Borough	0 non-null	float64
42	Taxi Pick Up Location	0 non-null	float64
43	Bridge Highway Name	297 non-null	str
44	Bridge Highway Direction	297 non-null	str
45	Road Ramp	262 non-null	str
46	Bridge Highway Segment	262 non-null	str
47	Garage Lot Name	0 non-null	float64
48	Ferry Direction	0 non-null	str
49	Ferry Terminal Name	0 non-null	str
50	Latitude	360470 non-null	float64

51 Longitude 360470 non-null float64
52 Location 360470 non-null str
dtypes: datetime64[us](2), float64(10), int64(1), str(40)
memory usage: 149.2 MB

In [13]: sr=sr[sr['Closed Date']>=sr['Created Date']]

In [14]: sr.head()

Out[14]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor	Location
0	32310363	2015-12-31 23:59:45	2016-01-01 00:55:15	NYPD	New York City Police Department	Noise - Street/Sidewalk	Loud Music/Party	Street/Side
1	32309934	2015-12-31 23:59:44	2016-01-01 01:26:57	NYPD	New York City Police Department	Blocked Driveway	No Access	Street/Side
2	32309159	2015-12-31 23:59:29	2016-01-01 04:51:03	NYPD	New York City Police Department	Blocked Driveway	No Access	Street/Side
3	32305098	2015-12-31 23:57:46	2016-01-01 07:43:13	NYPD	New York City Police Department	Illegal Parking	Commercial Overnight Parking	Street/Side
4	32306529	2015-12-31 23:56:58	2016-01-01 03:24:42	NYPD	New York City Police Department	Illegal Parking	Blocked Sidewalk	Street/Side

5 rows x 53 columns

In [15]: sr

Out[15]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor	Location
0	32310363	2015-12-31 23:59:45	2016-01-01 00:55:15	NYPD	New York City Police Department	Noise - Street/Sidewalk	Loud Music/Party	Stre
1	32309934	2015-12-31 23:59:44	2016-01-01 01:26:57	NYPD	New York City Police Department	Blocked Driveway	No Access	Stre
2	32309159	2015-12-31 23:59:29	2016-01-01 04:51:03	NYPD	New York City Police Department	Blocked Driveway	No Access	Stre
3	32305098	2015-12-31 23:57:46	2016-01-01 07:43:13	NYPD	New York City Police Department	Illegal Parking	Commercial Overnight Parking	Stre
4	32306529	2015-12-31 23:56:58	2016-01-01 03:24:42	NYPD	New York City Police Department	Illegal Parking	Blocked Sidewalk	Stre
...	...	...	...	...	...	...	...	...
364553	29609918	2015-01-01 00:04:44	2015-01-01 10:22:31	NYPD	New York City Police Department	Illegal Parking	Blocked Hydrant	Stre
364554	29608392	2015-01-01 00:04:28	2015-01-01 02:25:02	NYPD	New York City Police Department	Noise - Vehicle	Car/Truck Horn	Stre
364555	29607589	2015-01-01 00:01:30	2015-01-01 00:20:33	NYPD	New York City Police Department	Noise - Street/Sidewalk	Loud Music/Party	Stre
364556	29610889	2015-01-01 00:01:29	2015-01-01 02:42:22	NYPD	New York City Police Department	Blocked Driveway	No Access	Stre
364557	29611816	2015-01-01 00:00:50	2015-01-01 02:47:50	NYPD	New York City Police Department	Blocked Driveway	No Access	Stre

362177 rows × 53 columns



In [16]: `sr['Time_difference']=sr['Closed Date']-sr['Created Date']`

2.3.2 Convert the calculated date to seconds to get a better representation

In [17]: `sr['Time_Elapsed_seconds']=sr['Time_difference'].dt.total_seconds()`

```
In [18]: sr['Time_Elapsed_seconds'].head()
```

```
Out[18]: 0      3330.0
          1      5233.0
          2     17494.0
          3     27927.0
          4     12464.0
          Name: Time_Elapsed_seconds, dtype: float64
```

2.3.3 View the descriptive statistics for the newly created column

```
In [19]: sr['Time_Elapsed_seconds'].describe()
```

```
Out[19]: count      3.621770e+05
          mean      1.511330e+04
          std       2.110255e+04
          min       6.100000e+01
          25%       4.533000e+03
          50%       9.616000e+03
          75%       1.887800e+04
          max       2.134342e+06
          Name: Time_Elapsed_seconds, dtype: float64
```

```
In [20]: sr.head()
```

```
Out[20]:
```

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor	Location
0	32310363	2015-12-31 23:59:45	2016-01-01 00:55:15	NYPD	New York City Police Department	Noise - Street/Sidewalk	Loud Music/Party	Street/Side
1	32309934	2015-12-31 23:59:44	2016-01-01 01:26:57	NYPD	New York City Police Department	Blocked Driveway	No Access	Street/Side
2	32309159	2015-12-31 23:59:29	2016-01-01 04:51:03	NYPD	New York City Police Department	Blocked Driveway	No Access	Street/Side
3	32305098	2015-12-31 23:57:46	2016-01-01 07:43:13	NYPD	New York City Police Department	Illegal Parking	Commercial Overnight Parking	Street/Side
4	32306529	2015-12-31 23:56:58	2016-01-01 03:24:42	NYPD	New York City Police Department	Illegal Parking	Blocked Sidewalk	Street/Side

5 rows x 55 columns



2.3.4 Check the number of null values in the Complaint_Type and City columns

```
In [21]: sr['City']=sr['City'].isnull()  
sr['City']
```

```
Out[21]: 0      False  
1      False  
2      False  
3      False  
4      False  
...  
364553  False  
364554  False  
364555  False  
364556  False  
364557  False  
Name: City, Length: 362177, dtype: bool
```

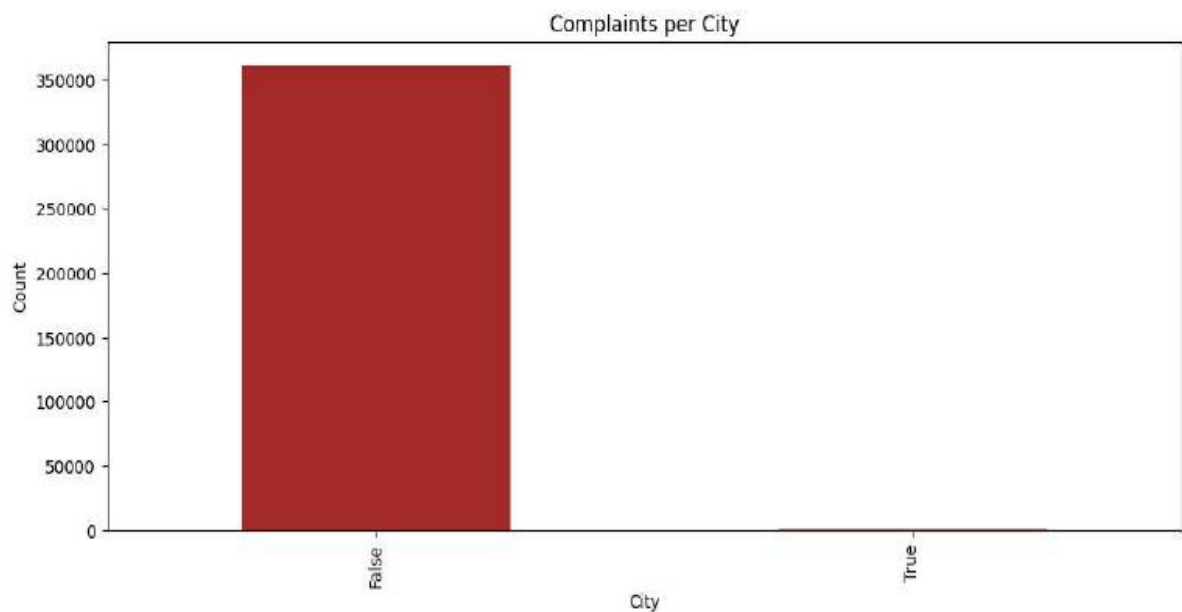
2.3.5 Impute the NA value with Unknown City

```
In [22]: sr['City']=sr['City'].fillna("Unknown City")
```

2.3.6 Draw a frequency plot for the complaints in each city

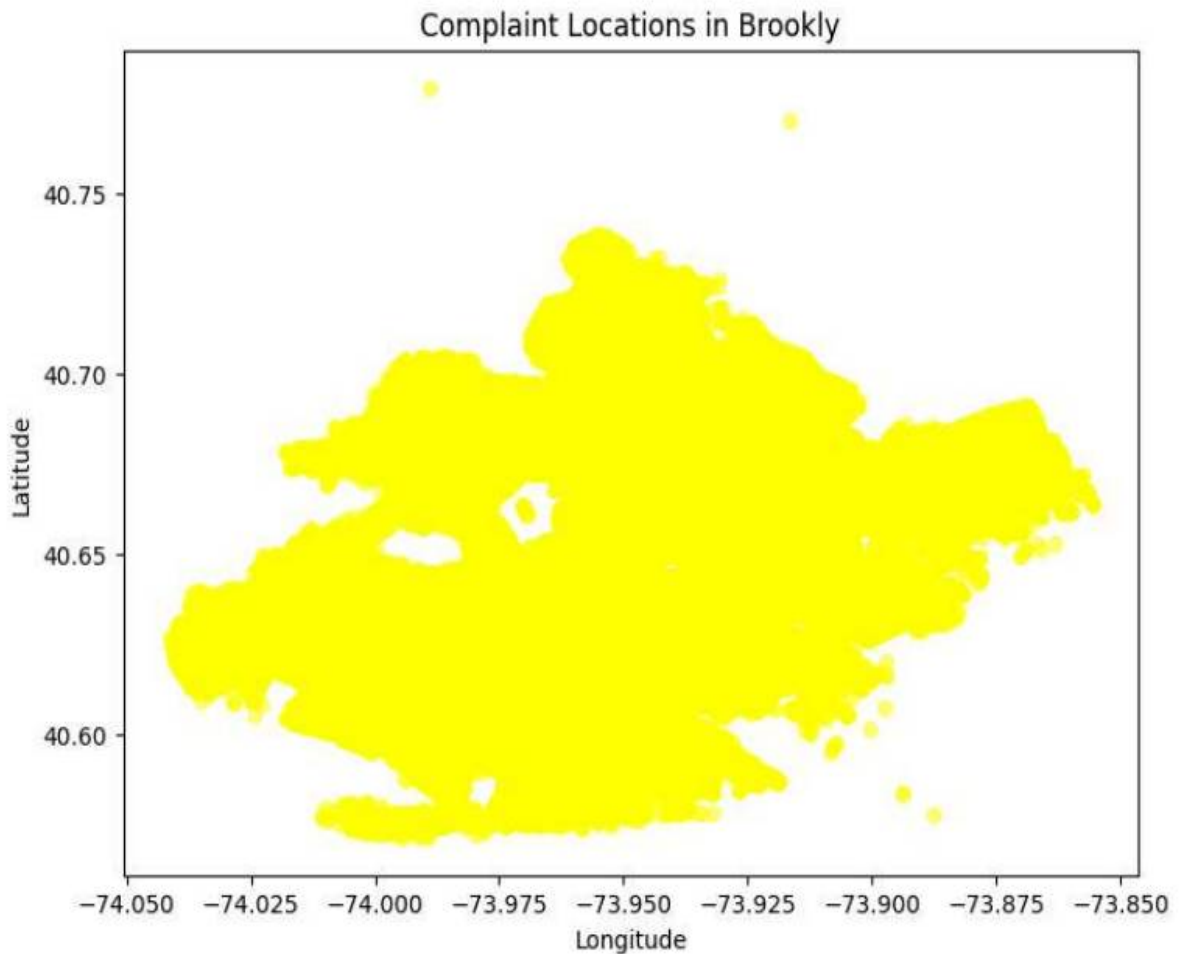
```
In [62]: sr['City'].value_counts().plot(kind='bar',color='brown',figsize=(12,5))  
plt.title('Complaints per City')  
plt.ylabel("Count")  
##plt.xlabel(rotation=90);
```

```
Out[62]: Text(0, 0.5, 'Count')
```

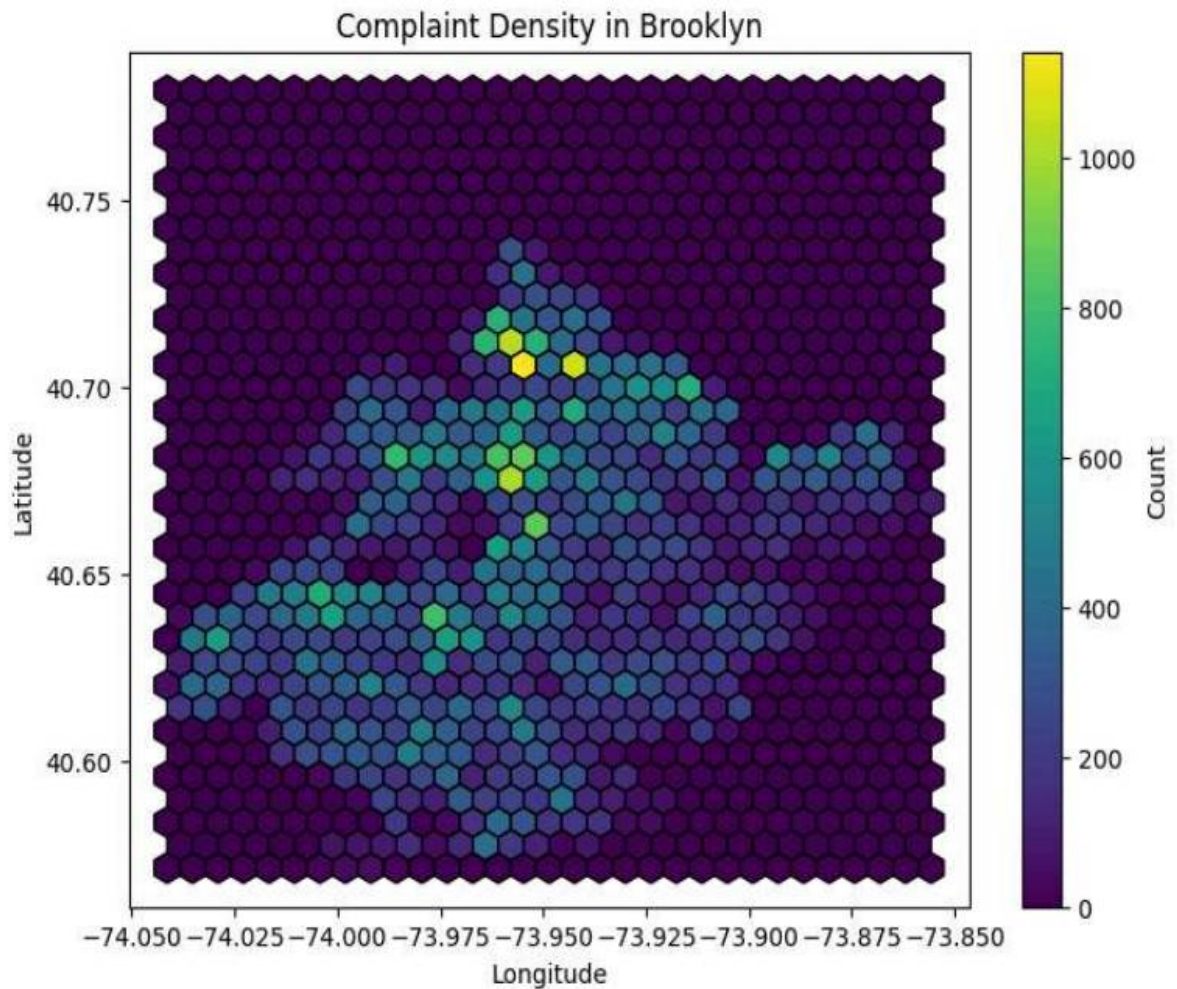


2.3.7 Create a scatter and hexbin plot of the concentration of complaints across Brooklyn

```
In [72]: brooklyn_df=sr[sr['City']=='BROOKLYN']  
#scatter plot(Longitude vs Latitude)  
plt.figure(figsize=(8,6))  
plt.scatter(brooklyn_df['Longitude'],brooklyn_df['Latitude'],color='yellow',alpha=0.5)  
plt.title("Complaint Locations in Brookly")  
plt.xlabel("Longitude")  
plt.ylabel("Latitude");
```



```
In [76]: #Hexbin plt  
plt.figure(figsize=(8,6))  
plt.hexbin(brooklyn_df['Longitude'],  
           brooklyn_df['Latitude'],color='black',gridsize=30)  
plt.colorbar(label='Count')  
plt.title('Complaint Density in Brooklyn')  
plt.xlabel("Longitude")  
plt.ylabel("Latitude")  
plt.show()
```



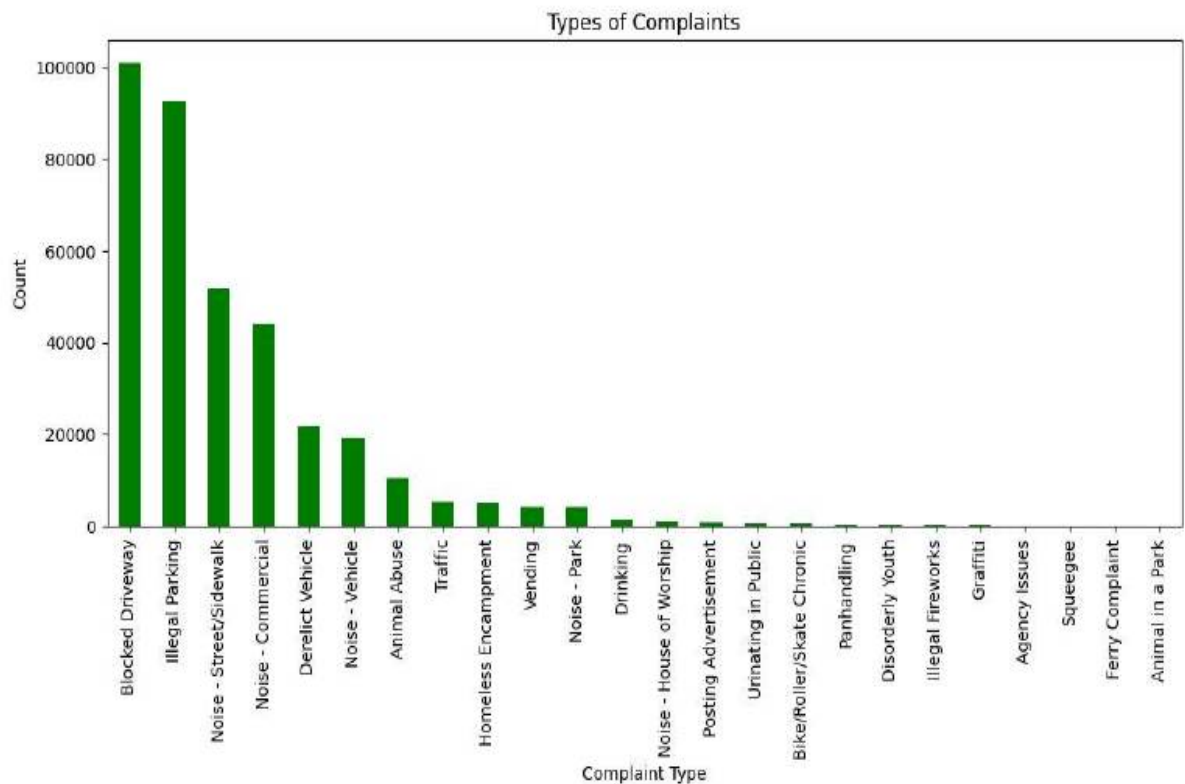
In []:

In []:

3. Find major types of complaints:

3.1 Plot a bar graph to show the types of complaints

```
In [77]: sr['Complaint Type'].value_counts().plot(kind='bar', color='green', figsize=(12,5))
plt.title("Types of Complaints")
plt.ylabel("Count")
plt.xticks(rotation=90)
plt.show()
```



3.2 Check the frequency of various types of complaints for New York City

```
In [27]: nyc_df = sr[sr['City'] == 'NEW YORK']
nyc_df['Complaint Type'].value_counts()
```

```
Out[27]: Series([], Name: count, dtype: int64)
```

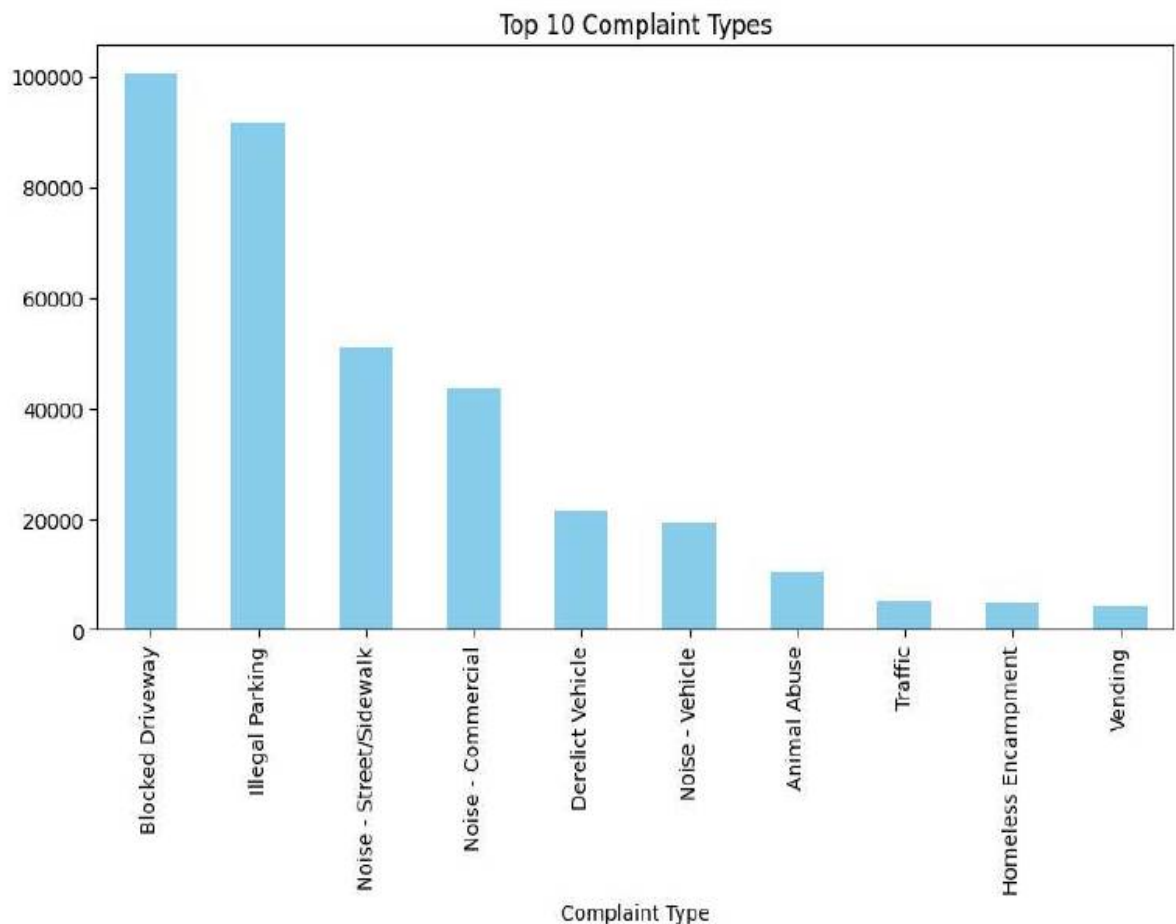
3.3 Find the top 10 complaint types

```
In [28]: top10 = sr['Complaint Type'].value_counts().head(10)
print(top10)
```

```
Complaint Type
Blocked Driveway      100624
Illegal Parking        91716
Noise - Street/Sidewalk 51139
Noise - Commercial    43751
Derelict Vehicle      21518
Noise - Vehicle       19301
Animal Abuse         10530
Traffic               5196
Homeless Encampment   4879
Vending               4185
Name: count, dtype: int64
```

3.4 Display the various types of complaints in each city

```
In [80]: top10.plot(kind='bar', color='skyblue', figsize=(10,5))  
plt.title("Top 10 Complaint Types")  
plt.show()
```



3.5 Create a DataFrame, df_new, which contains cities as columns and complaint types in rows

```
In [30]: sr.groupby('City')['Complaint Type'].value_counts()
```

```

Out[30]: City Complaint Type
False Blocked Driveway 100538
        Illegal Parking 91404
        Noise - Street/Sidewalk 51040
        Noise - Commercial 43672
        Derelict Vehicle 21455
        Noise - Vehicle 19292
        Animal Abuse 10529
        Traffic 5194
        Homeless Encampment 4878
        Vending 4184
        Noise - Park 4082
        Drinking 1401
        Noise - House of Worship 1068
        Posting Advertisement 679
        Urinating in Public 641
        Bike/Roller/Skate Chronic 473
        Panhandling 324
        Disorderly Youth 315
        Illegal Fireworks 172
        Graffiti 157
        Squeegee 4
        Animal in a Park 1
True Illegal Parking 312
      Noise - Street/Sidewalk 99
      Blocked Driveway 86
      Noise - Commercial 79
      Derelict Vehicle 63
      Noise - Vehicle 9
      Agency Issues 8
      Noise - Park 7
      Drinking 3
      Bike/Roller/Skate Chronic 2
      Traffic 2
      Vending 1
      Animal Abuse 1
      Homeless Encampment 1
      Panhandling 1
Name: count, dtype: int64

```

```

In [31]: #for better view
sr.groupby('City')['Complaint Type'].value_counts().unstack().fillna(0)

```

Out[31]:

Complaint Type	Agency Issues	Animal Abuse	Animal in a Park	Bike/Roller/Skate Chronic	Blocked Driveway	Derelict Vehicle	Disorderly Youth	Dri
City								
False	0.0	10529.0	1.0	473.0	100538.0	21455.0	315.0	
True	8.0	1.0	0.0	2.0	86.0	63.0	0.0	

2 rows × 23 columns



```
In [32]: #pivot Table
df_new = pd.crosstab(sr['Complaint Type'], sr['City'])
```

```
In [33]: df_new.head()
```

Out[33]:

	City	False	True
Complaint Type			
Agency Issues		0	8
Animal Abuse		10529	1
Animal in a Park		1	0
Bike/Roller/Skate Chronic		473	2
Blocked Driveway		100538	86

```
In [34]: #Alternative
df_new2 = sr.pivot_table(index='Complaint Type', columns='City', aggfunc='size', fi
```

```
In [35]: df_new2.head()
```

Out[35]:

	City	False	True
Complaint Type			
Agency Issues		0	8
Animal Abuse		10529	1
Animal in a Park		1	0
Bike/Roller/Skate Chronic		473	2
Blocked Driveway		100538	86

```
In [36]: #I used value_counts for frequency analysis, groupby for city-level insights, and c
#to transform data into a matrix format for better comparison across cities.
```

In []:

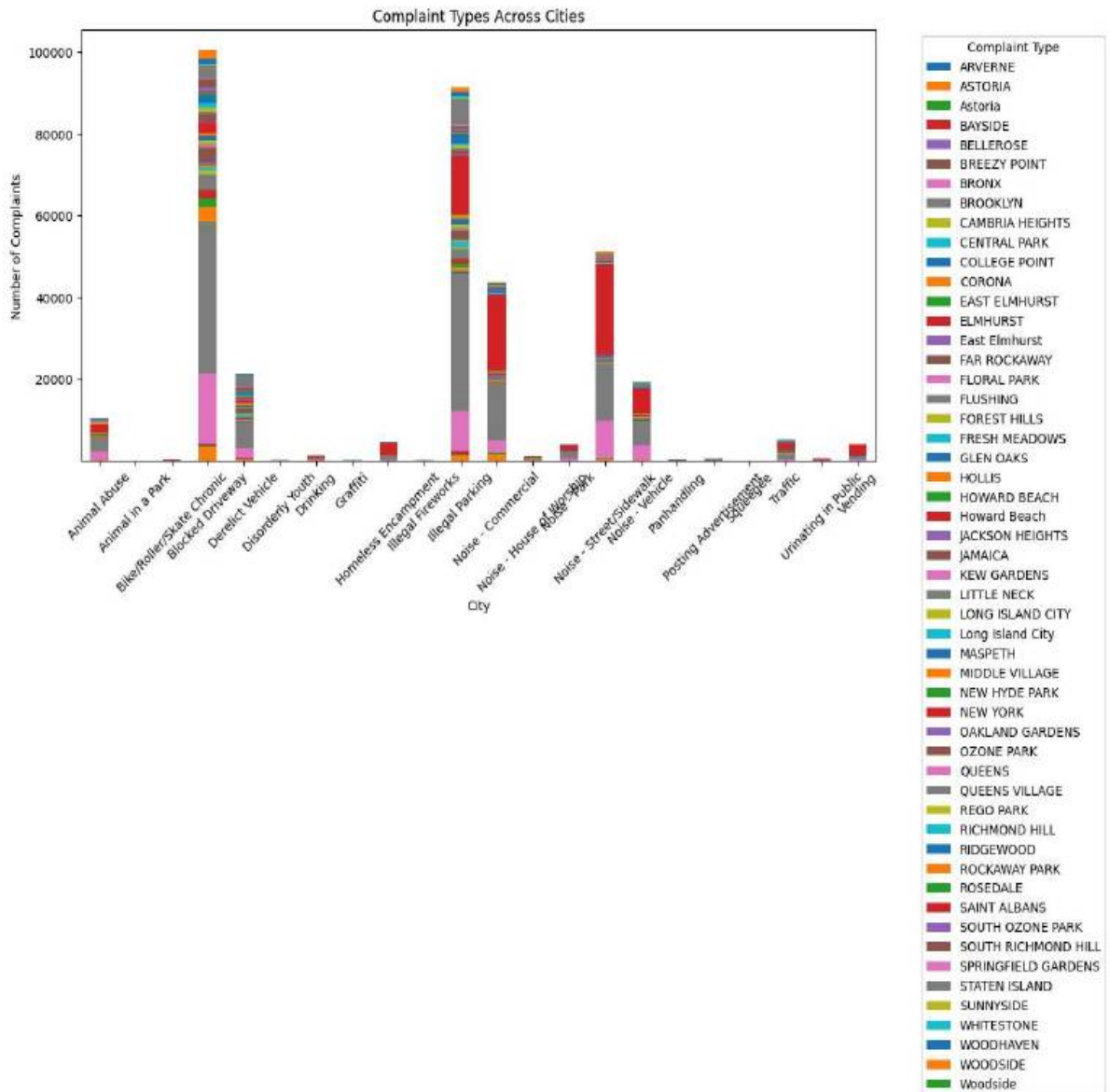
4. Visualize the major types of complaints in each city

4.1 Draw another chart that shows the types of complaints in each city in a single chart, where different colors show the different types of complaints

```
In [82]: df_new = sr.pivot_table(index='Complaint Type', columns='City', aggfunc='size', fill_value=0)

df_new.plot(kind='bar', stacked=True, figsize=(12,6))

plt.title("Complaint Types Across Cities")
plt.xlabel("City")
plt.ylabel("Number of Complaints")
plt.xticks(rotation=45)
plt.legend(title="Complaint Type", bbox_to_anchor=(1.05,1))
plt.show()
```

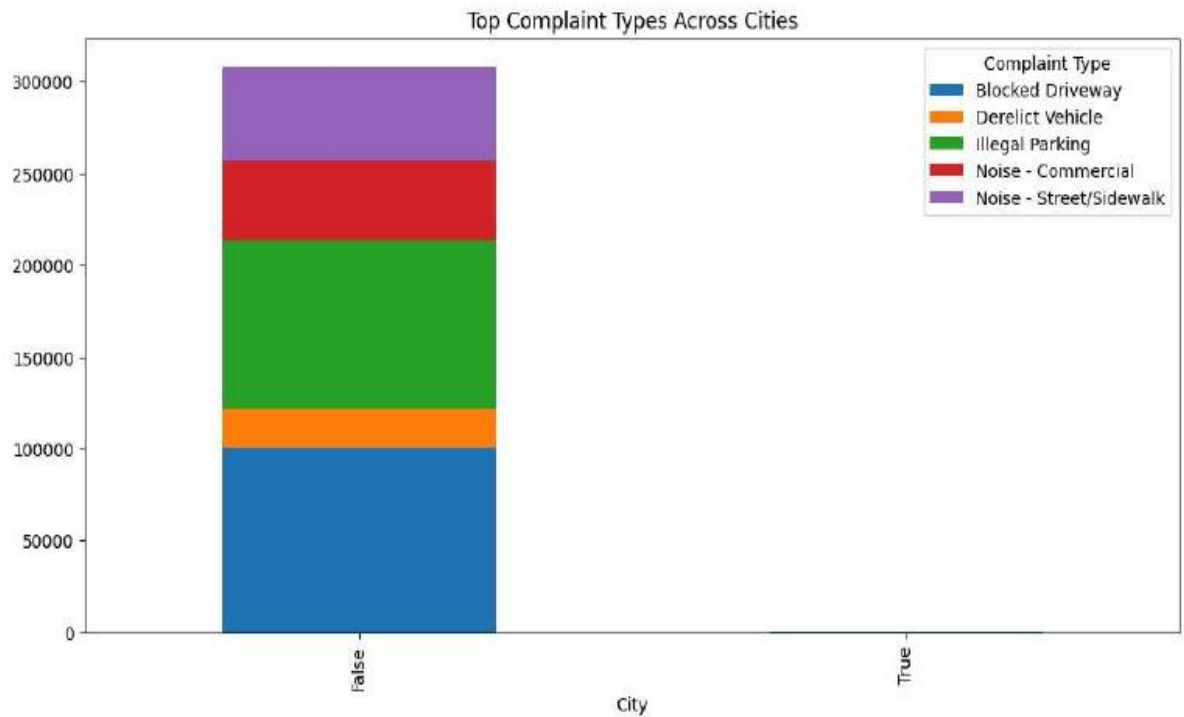


```
In [38]: top_complaints = sr['Complaint Type'].value_counts().head(5).index

df_filtered = sr[sr['Complaint Type'].isin(top_complaints)]

df_plot = pd.crosstab(df_filtered['City'], df_filtered['Complaint Type'])

df_plot.plot(kind='bar', stacked=True, figsize=(12,6))
plt.title("Top Complaint Types Across Cities")
plt.show()
```



```
In [39]: avg_time = sr.groupby(['City', 'Complaint Type'])['Time_Elapsed_seconds'].mean().re
```

4.2 Sort the complaint types based on the average Request_Closing_Time grouping them for different locations

```
In [40]: avg_time_sorted = avg_time.sort_values(by='Time_Elapsed_seconds', ascending=False)
avg_time_sorted.head(10)
```

```
Out[40]:
```

	City	Complaint Type	Time_Elapsed_seconds
1	False	Animal in a Park	1.212634e+06
26	True	Derelict Vehicle	5.251333e+04
34	True	Panhandling	4.433900e+04
30	True	Noise - Commercial	3.537128e+04
29	True	Illegal Parking	3.322397e+04
25	True	Blocked Driveway	3.224407e+04
35	True	Traffic	3.141700e+04
31	True	Noise - Park	3.091329e+04
4	False	Derelict Vehicle	2.527987e+04
32	True	Noise - Street/Sidewalk	2.470965e+04

```
In [41]: # I used stacked bar charts to compare complaint distribution across cities and gro
```

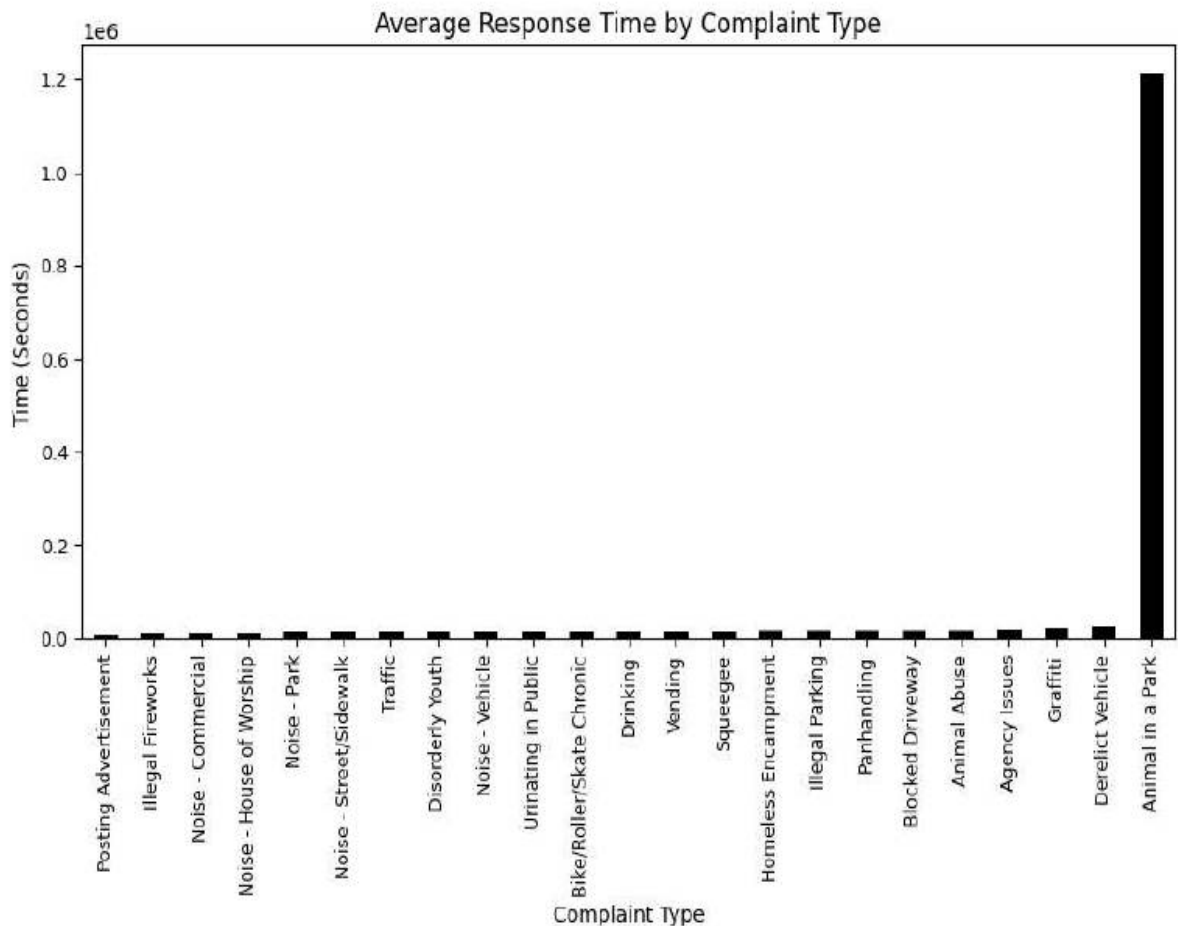
5. See whether the average response time across different complaint types is similar (overall)

5.1 Visualize the average of Request_Closing_Time

```
In [42]: avg_response = sr.groupby('Complaint Type')['Time_Elapsed_seconds'].mean().sort_val  
avg_response.head()
```

```
Out[42]: Complaint Type  
Posting Advertisement      7286.256259  
Illegal Fireworks         10113.482558  
Noise - Commercial        11085.760531  
Noise - House of Worship  11391.087079  
Noise - Park              12226.055515  
Name: Time_Elapsed_seconds, dtype: float64
```

```
In [83]: plt.figure(figsize=(12,6))  
avg_response.plot(kind='bar',color='black',fontsize=9,figsize=(10,5))  
  
plt.title("Average Response Time by Complaint Type")  
plt.ylabel("Time (Seconds)")  
plt.xticks(rotation=90)  
plt.show()
```



In []:

6. Identify the significant variables by performing statistical analysis using p-values

In [44]: `from scipy import stats`

In [45]: `df_test = sr[['Complaint Type', 'Time_Elapsed_seconds']].dropna()`

In [46]: `groups = [
 group['Time_Elapsed_seconds'].values
 for name, group in df_test.groupby('Complaint Type')
]`

In [47]: `f_stat, p_value = stats.f_oneway(*groups)`

```
print("F-statistic:", f_stat)
print("P-value:", p_value)
```

F-statistic: 565.261570041763

P-value: 0.0

In [48]: `# Complaint Type is significant (affects response time) because p value < 0.5`

```
In [49]: # "I performed ANOVA to test whether the mean response time differs across complain
```

7. Perform a Kruskal-WallisH test

7.1 Fail to reject H0: All sample distributions are equal

7.2 Reject H0: One or more sample distributions are not equal

```
In [50]: df_test = sr[['Complaint Type', 'Time_Elapsed_seconds']].dropna()
```

```
In [51]: top5 = sr['Complaint Type'].value_counts().head(5).index
df_test = df_test[df_test['Complaint Type'].isin(top5)]
```

```
In [52]: groups = [
    group['Time_Elapsed_seconds'].values
    for name, group in df_test.groupby('Complaint Type')
]
```

```
In [53]: h_stat, p_value = stats.kruskal(*groups)

print("H-statistic:", h_stat)
print("P-value:", p_value)
```

```
H-statistic: 10028.143068069874
P-value: 0.0
```

```
In [ ]:
```

```
In [54]: p_value < 0.05
```

```
Out[54]: np.True_
```

```
In [55]: #Reject H0: One or more sample distributions are not equal"
```

```
In [56]: # # Since the data was not normally distributed, I used the Kruskal-Wallis test. Ba
# complaint types have significantly different response time distributions
```

8. Present your observations

```
In [57]: # The analysis of the complaint dataset reveals that both complaint type and city L

# A small number of complaint types dominate the dataset, indicating recurring urba
# Complaint distribution varies across cities, with high-density areas like New Yor
# The response time is not uniform; certain complaint types take longer to resolve,
```

Visualization techniques highlighted geographical hotspots, helping identify area
Statistical analysis using ANOVA and the Kruskal-Wallis H test confirmed that the